

HAL CORE ENGINEERING · PREDICTIVE & ANALYTICS TRACK

Predictive AI a field guide.

A keep-it reference to everything you
ran in the hands-on lab — the ideas, the
pictures, and where they fit your work.

Text

Sound

Speech

No maths to memorise

One idea, two kinds of messy data — written text & spoken speech · four
hands-on sessions · written for engineers, not programmers

01 What predictive AI actually is

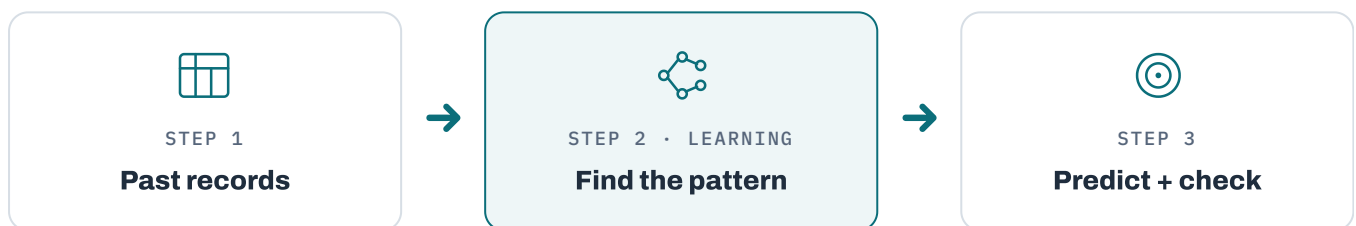
Strip away the buzzwords and predictive AI is one sentence:

Learn the pattern from past examples, then predict the next case.

A model studies many records where the answer is already known, finds the pattern that connects the inputs to the answer, and uses that pattern to make a call on a new case it has never seen. Like a seasoned technician who has inspected thousands of machines and can spot a bad one — not magic, just **pattern from experience**.

The five-step rhythm behind every session

Every notebook you ran — whether the data was written text, machine sound, or spoken speech — followed the same loop. You met it in Session 1 and reused it all day.



In full, the five words you will keep hearing: **dataset** → **split** → **train** → **predict** → **measure**.

The words that sound scary (and what they really mean)

Dataset — a table; rows are cases, columns are measurements.

Feature — an input column; a clue the model gets to see.

Label / label column — the answer we want, and the column that holds it.

Model — the pattern-finder; "training" is when it learns.

Why we hide some data — the train/test split

We never judge a model on the data it studied — that would be like grading a student on the exact exam paper they revised from. We hold back a slice and score the model only on those unseen cases.

75%

TRAINING SET — THE MODEL LEARNS FROM THIS

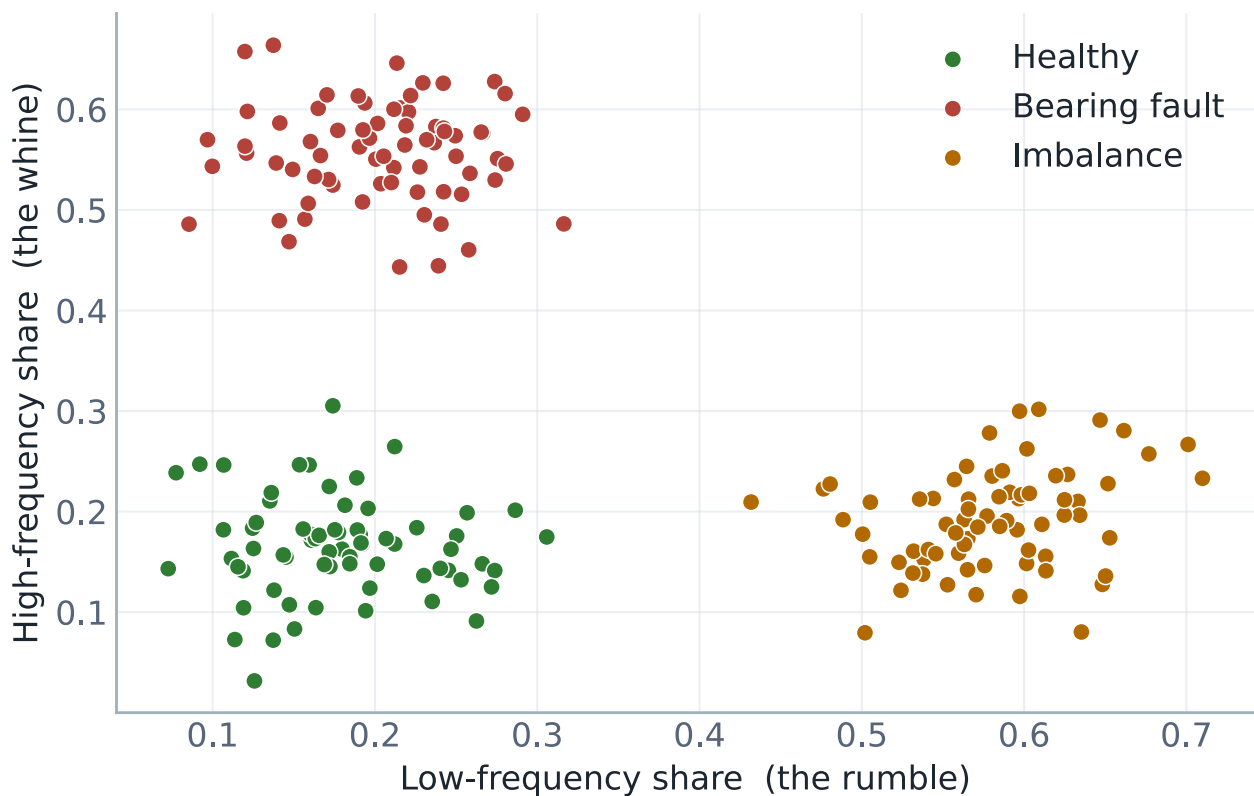
25%

TEST SET — HIDDEN, FOR AN HONEST SCORE

Accuracy is then simply: out of those hidden cases, how many did it get right? A model that scores 100% on real data is usually cheating, not winning.

02 A model draws a boundary

Here is the single most useful picture from the day. Each dot is a machine, placed by the **sound it made** — how much of its noise is low-frequency rumble (left to right) versus high-frequency whine (bottom to top). Healthy, bearing-fault and imbalance machines fall in different regions, and the model's whole job is to learn the boundaries between them.



Each dot is one recording. Healthy, bearing and imbalance faults separate by their sound alone.

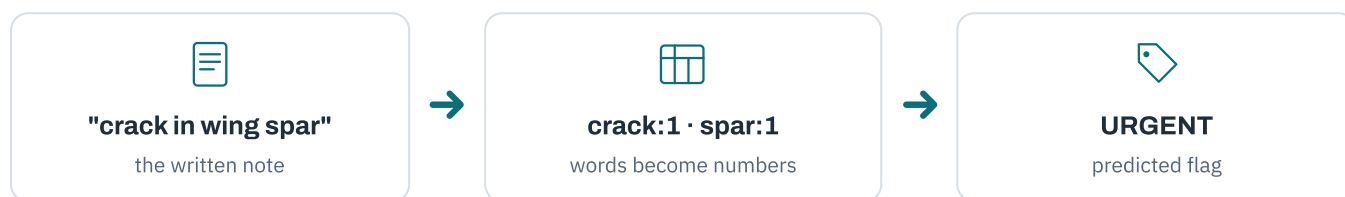
When a brand-new recording arrives, the model simply checks **which region** its rumble-and-whine numbers fall in, and predicts that fault. The very same picture works for written reports and spoken notes too — the only thing that changes between sessions is **how we turn the raw data into numbers** in the first place.

Takeaway: the model isn't memorising machines — it's learning a rule that separates the groups, so it can judge cases it has never seen.

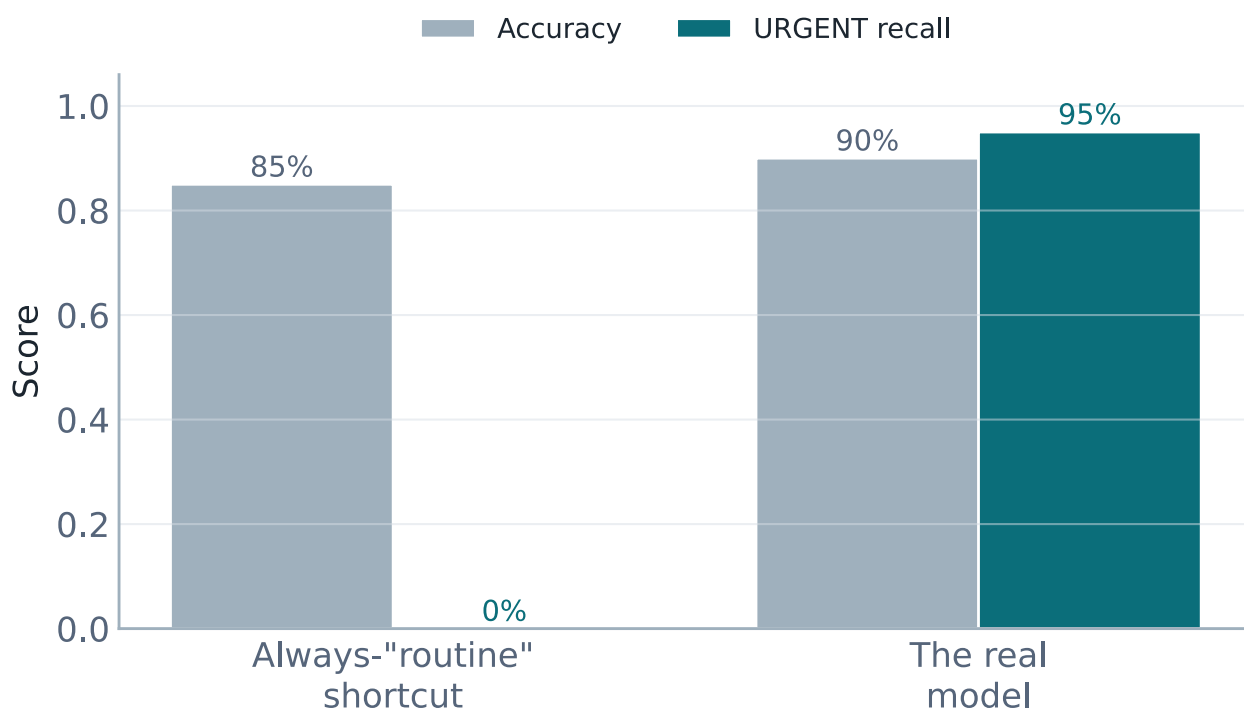
S1 Text I — flag the urgent reports

Scenario: a maintenance desk gets hundreds of short free-text notes a day — "crack found in wing spar, aircraft grounded", "cabin light flickers, cosmetic only". Almost all are routine; a few are **urgent** and must not sit in a queue overnight. The job: flag the urgent ones the moment they arrive.

The one new idea — and the foundation for the whole day: computers can't read words, only numbers. So we first turn each note into a count of which words it contains, letting rare, telling words count for more and ignoring filler like "the" (this is **TF-IDF**). After that, it is the five-step rhythm — and everything from here just changes how we make the numbers.



The honesty lesson of the day — the accuracy trap: when urgent notes are rare, a lazy model that always says "routine" scores about **85% accuracy** while catching *zero* urgent notes. So we look past accuracy to **URGENT recall** — of all the truly urgent notes, how many did it catch? The real model reached ~90% accuracy and caught about **95%** of the urgent ones.

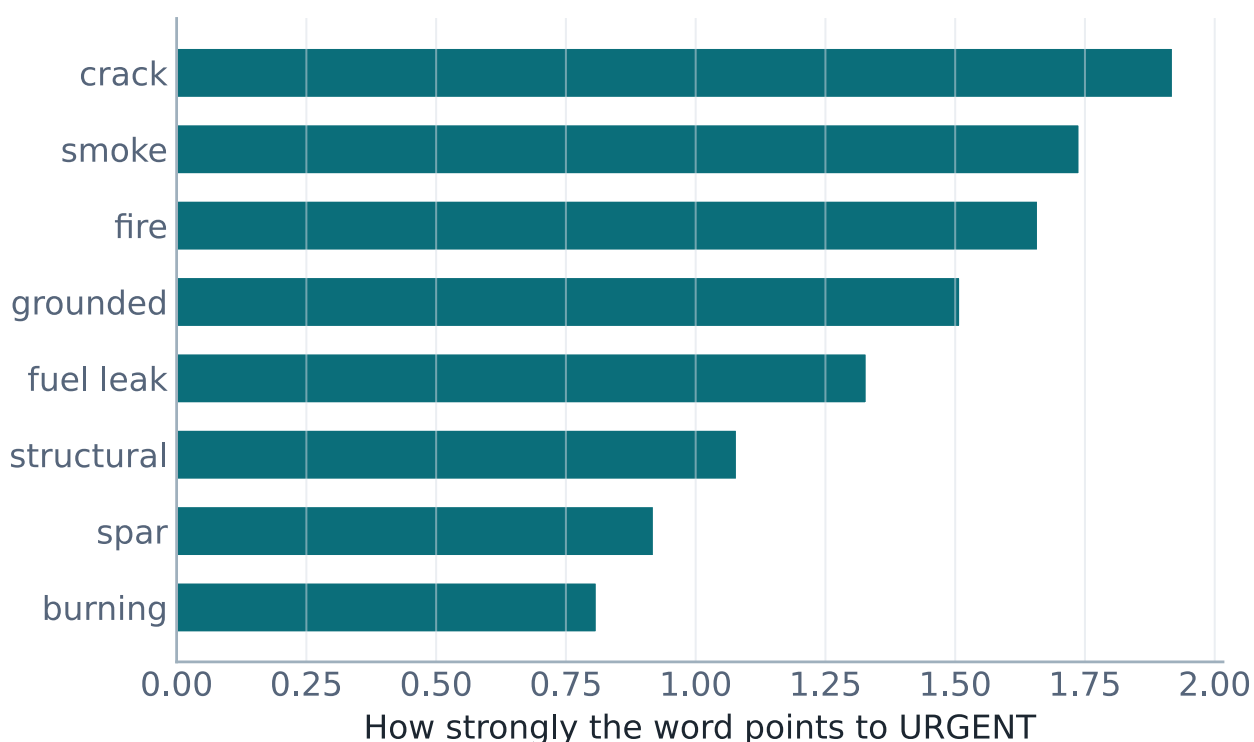


The always-"routine" shortcut looks 85% accurate yet catches nothing. The real model: ~90% accurate, ~95% urgent recall.

The **confusion matrix** is what tells these apart — and the dangerous box is the one a single accuracy number hides:

	MODEL SAID ROUTINE	MODEL SAID URGENT
TRULY ROUTINE	Correct left alone, rightly	False alarm routine, flagged
TRULY URGENT	Missed urgent, slipped through	Caught urgent, surfaced

It also showed its **clue-words** — the words that most push a note toward URGENT — and they read like plain English, exactly what a human would watch for:



crack, smoke, fire, grounded... — the words the model leaned on to call a note urgent.

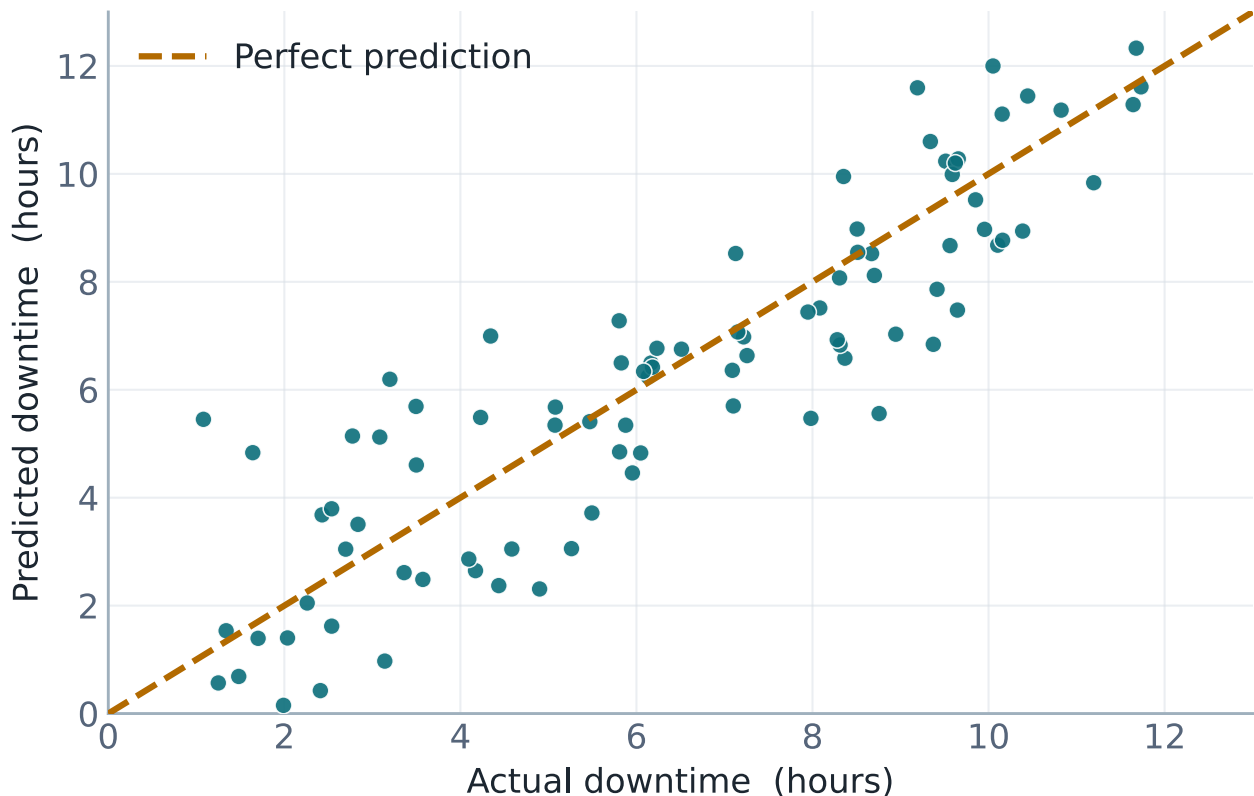
Takeaway: on rare-but-critical cases, a high accuracy can hide a useless model. Ask what it *misses*, not just how often it's right.

Where it fits: triaging snags, defect reports and complaints so the few that cannot wait surface first — and, in plain words, *why* each was flagged.

s2 Text II — predict, extract & find recurring faults

Scenario: the same pile of written snags, now asked harder questions than "which bucket?". Once words are numbers, the same machinery answers what a maintenance manager actually wants.

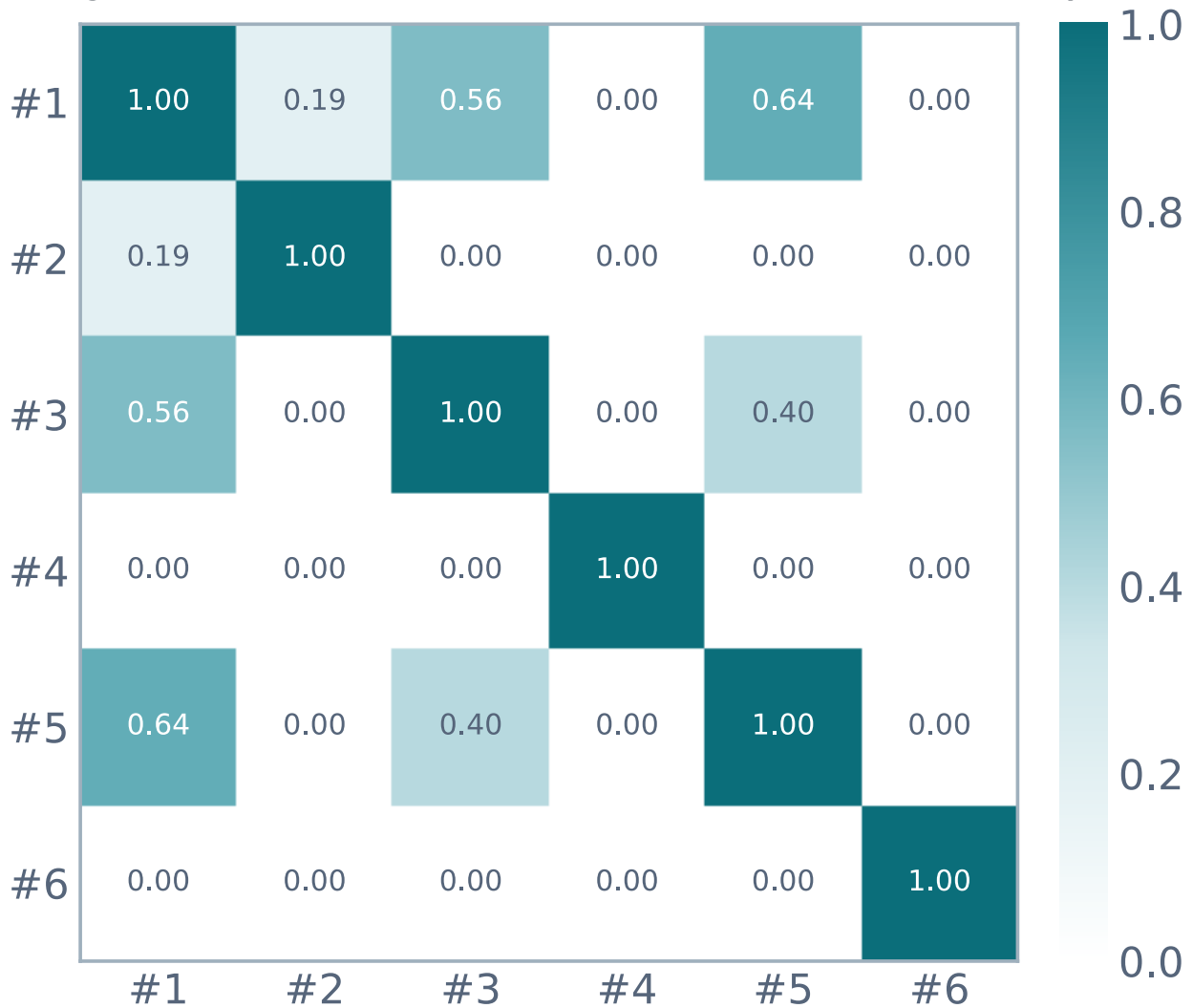
Predict a number, not just a category. From the wording alone, estimate the likely **downtime in hours** — that is **regression** (a number on a scale) rather than classification. Honest, not perfect: predictions land typically within about **1.5 hours** of the truth.



Predicted vs actual repair hours. Close to the dashed line is good; the spread is the honest error.

Find the same fault, worded five different ways. Chronic defects get logged again and again in different words. **Cosine similarity** measures how alike two reports are once they are numbers — with no labels at all — so near-duplicates light up automatically.

Snags #1, #3, #5 are the same fault, worded differently



Snags #1, #3, #5 are one chronic fault, worded three ways – they light up as near-duplicates.

The session also **grouped snags into families** with no labels (embeddings + **clustering**) – honest about its limits, the agreement with the true systems was a modest **~0.22**: some families pop out cleanly, others blend. And it **pulled part numbers** out of the prose into an auditable list a human can check line by line.

Takeaway: free-text logs are rich data. The same turn-words-into-numbers move predicts quantities, finds repeats, groups themes and extracts facts — supervised or not.

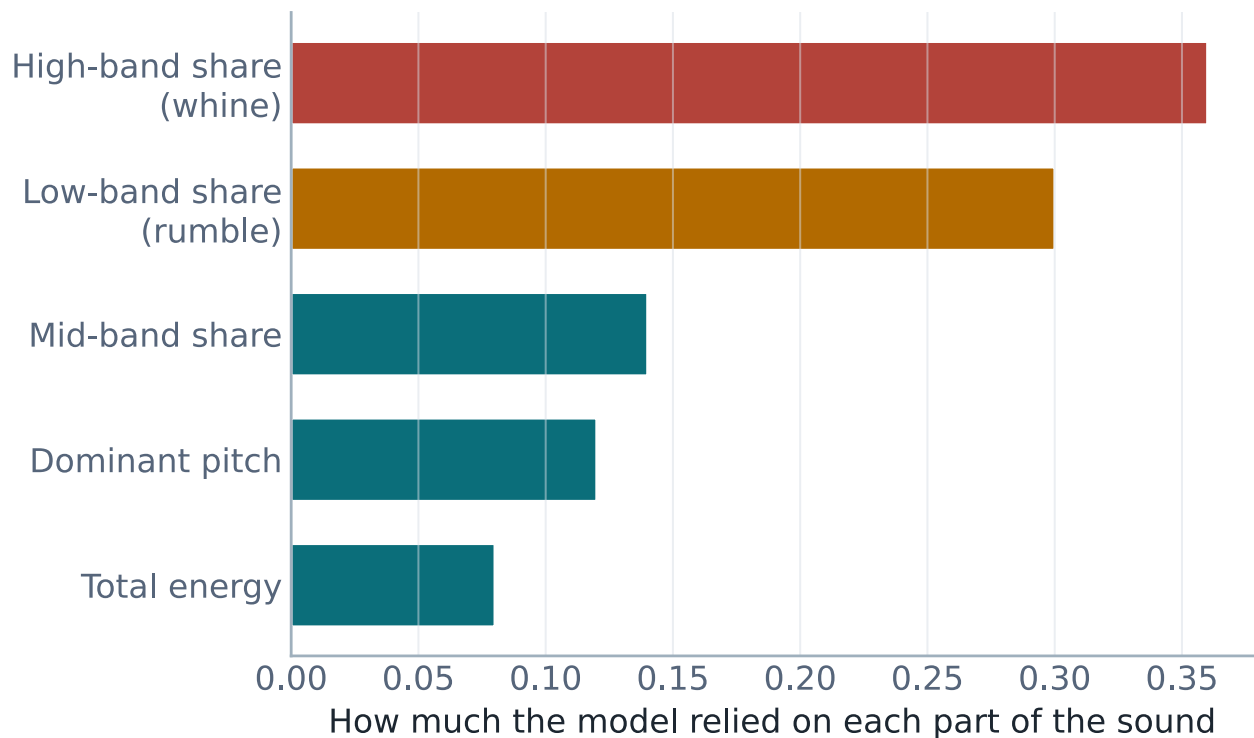
Where it fits: estimating repair effort, catching chronic recurring defects, auto-organising a logbook, lifting part/serial numbers out of reports.

s3 Speech I — predicting from the sound

Scenario: forget the words entirely — listen to a machine. From a short recording of a bearing, gearbox or fan, predict whether it is healthy or which fault it has, the way an experienced technician calls a bad machine by ear.

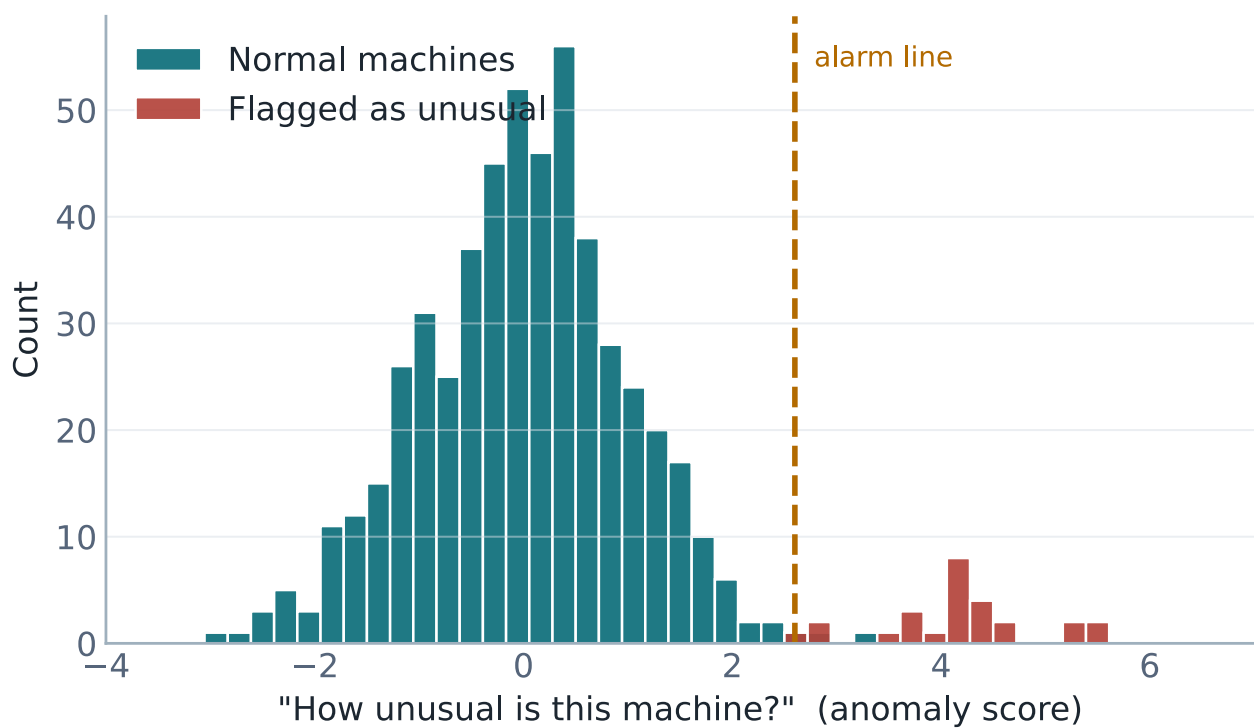
The one new idea: a sound is already numbers. The **FFT** (a fast bit of maths) splits a recording into its pitches, like a prism splitting light — how much rumble, how much whine. A handful of those become the feature columns, and then it is the same five steps. Three fault types separate cleanly (the picture in Part 2), and the model classified them at about **0.88** accuracy.

It tells you which part of the sound it used — feature importance — so you can check the machine's verdict against engineering intuition.



The model leaned on the whine and the rumble — the very cues a technician's ear already uses.

And when there are no fault examples at all — the realistic case in aerospace, where most machines are healthy — it switches to **anomaly detection**: learn only what "normal" sounds like, then flag the few recordings that don't fit (about **0.94** AUC, a strong-but-honest score).



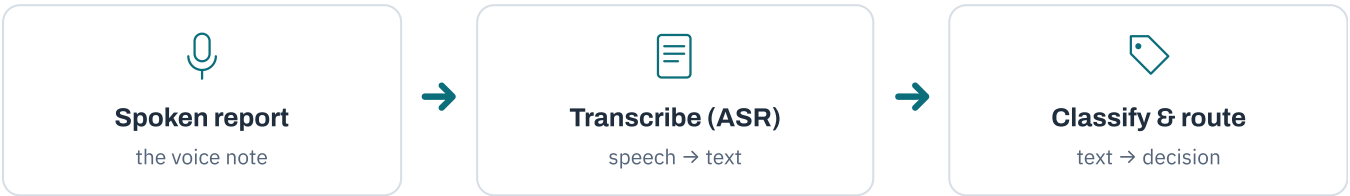
Learn what "normal" sounds like, then flag the few machines that don't fit – no fault labels needed.

Takeaway: acoustic monitoring is real predictive AI — it gives a verdict *and* the reason, and it can flag "odd ones out" with no failure history at all.

Where it fits: condition monitoring of bearings, gearboxes and engines; prioritising which equipment to inspect; catching machines drifting away from normal.

Scenario: a technician on the floor would rather **speak** a quick report than stop and type it. The computer listens, writes it down, and routes it — hands-free.

Speech analytics is just two familiar steps chained: **speech-to-text** (ASR — automatic speech recognition) turns the voice into words, and then it is the text analytics from the morning again. Audio in, text out, then a decision.



The honesty that makes it believable — Word Error Rate (WER): the speech version of accuracy. A low error rate is good, but a "cleaner" transcript can still get the **one word that matters** wrong — a digit in a part number — which is why numbers and serials always get a human's eyes.

WHAT THE COMPUTER HEARD	WORD ERROR RATE	PART NUMBER
clean studio voice	0%	correct
shop-floor noise	15%	correct
misheard a single digit	8%	WRONG

ASR is hard by default (accents, jargon, radio noise) but strong with domain data — air-traffic systems reach **>95%** on controllers and ~97% on call-signs after training. For high-stakes commands, a small fixed **keyword** vocabulary is more reliable than full dictation.

Takeaway: speech = speech-to-text + text analytics. The same one idea, now reaching the spoken word — with its limits measured, not hidden.

Where it fits: verbal handovers, inspection voice memos and call logs — information that today is spoken once and then lost.

04 Powerful, not magic

The most useful thing you can take away is a clear sense of where this technology helps and where it doesn't. Four limits worth remembering:

Only as good as the data

It learns from the records you feed it. Biased or thin history gives biased or thin predictions.

Never perfect

Real data surprises. We measure the gap honestly — a missed urgent note, a misheard digit — rather than hide it.

It assists; you decide

It ranks and flags *with a reason*. The engineer still makes the call.

Predict, not create

This is the predictive half of AI. The "generative" half (ChatGPT-style) is a different tool for a different job.

Full glossary

Dataset — a table of cases (rows) and measurements (columns).

Label column — the column holding the answer to predict.

Test set — hidden rows kept for an honest score.

Classification — predicting a category (urgent / routine).

Confusion matrix — the table of hits, false alarms and missed cases.

TF-IDF (vectorising) — turning words into numbers, weighting telling words.

Clustering — grouping cases into families with no labels.

Feature importance — which signal the model leaned on most.

Feature — an input column the model uses as a clue.

Training set — the rows the model learns from.

Accuracy — fraction of unseen cases it gets right.

Regression — predicting a number on a scale (downtime hours).

Precision / recall — how often a flag is right / how many real cases caught.

Cosine similarity — how alike two reports are once they are numbers.

FFT / spectrogram — splitting a sound into its pitches over time.

Anomaly detection — flagging unusual cases with no labels.

Supervised / unsupervised — learning with answers
/ finding structure without them.

Word Error Rate (WER) — how much a transcript got
wrong; speech's "accuracy".

Speech-to-text (ASR) — turning the spoken word into
text.

Keyword spotting — listening for a small fixed set of
commands, reliably.

Run it again — and what to try

Every notebook opens in **Google Colab** from a web browser, with nothing to install. Ask the facilitator for the link, or open colab.research.google.com and use **File → Upload notebook**. The best way to build intuition is to **change one value and re-run** — edit a note's wording, hide more of the data, or swap the spoken sentence — and watch the prediction move. Bring a problem from your own work: if you can phrase it as "from these inputs, predict that outcome," it may well be a predictive-AI problem.